

Programação Orientada a Objetos

Professor: Carlos Sabino



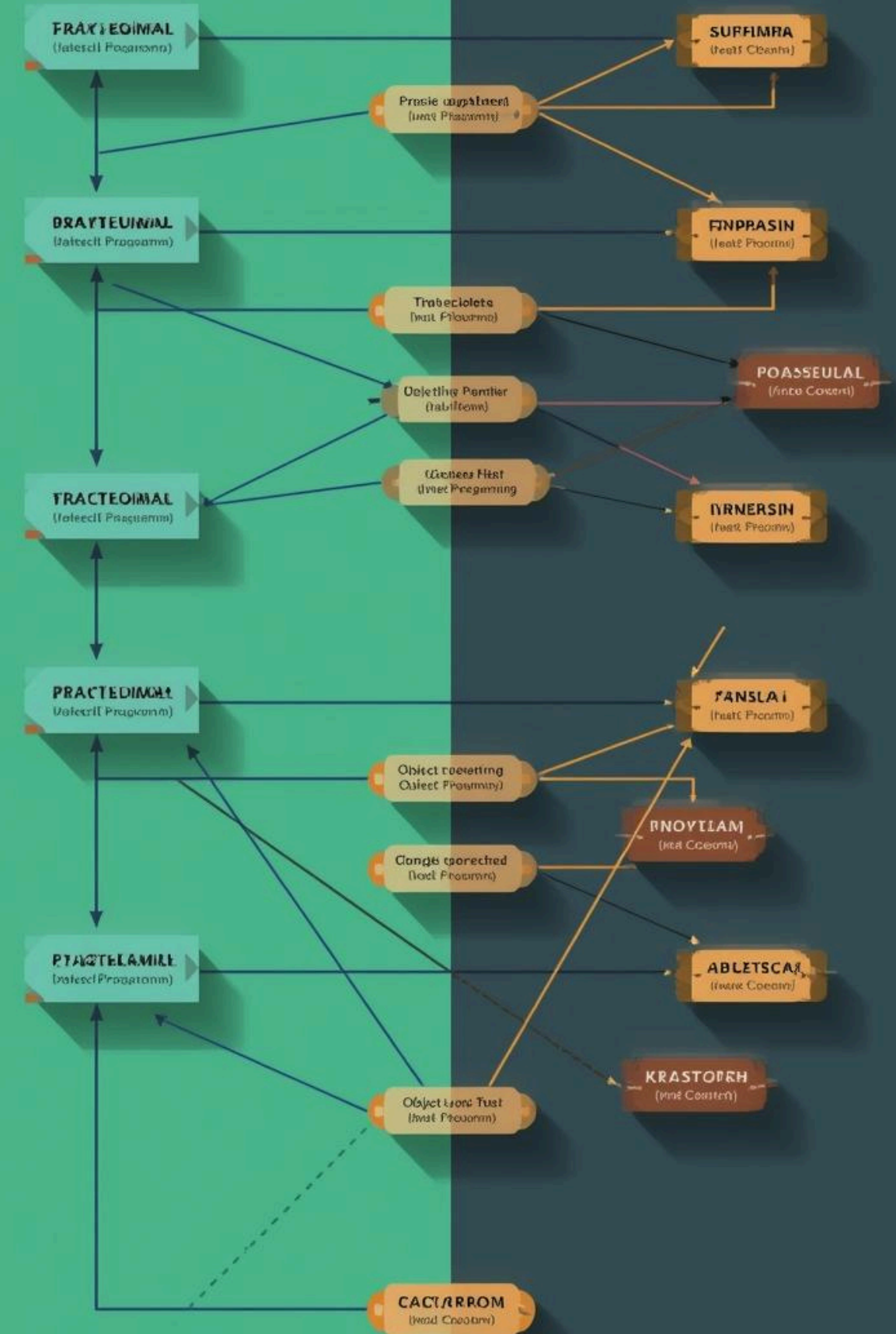
Paradigmas de Programação

Um paradigma de programação é um modelo ou estilo de programação que define como o programador organiza e estrutura o código.

PROCEDURAL
PROGRAMMING

VS.

OBJECT-ORIENTED
PROGRAMMING



Introdução a POO

O que é POO ?

Programação Orientada a Objetos (POO) é um paradigma de programação que busca representar elementos do mundo real no ambiente computacional por meio de objetos.

Esses objetos combinam dados (atributos) e comportamentos (métodos), permitindo que o sistema seja construído de forma mais modular, reutilizável e próxima da realidade.

Base class

```
{ Chase-geetv: 4}
Chase-petical )
Chase-ceal: }
cocnestc: }
fotlere: detien((tegarallving())| ) ) )
} )
```

Catine Classe

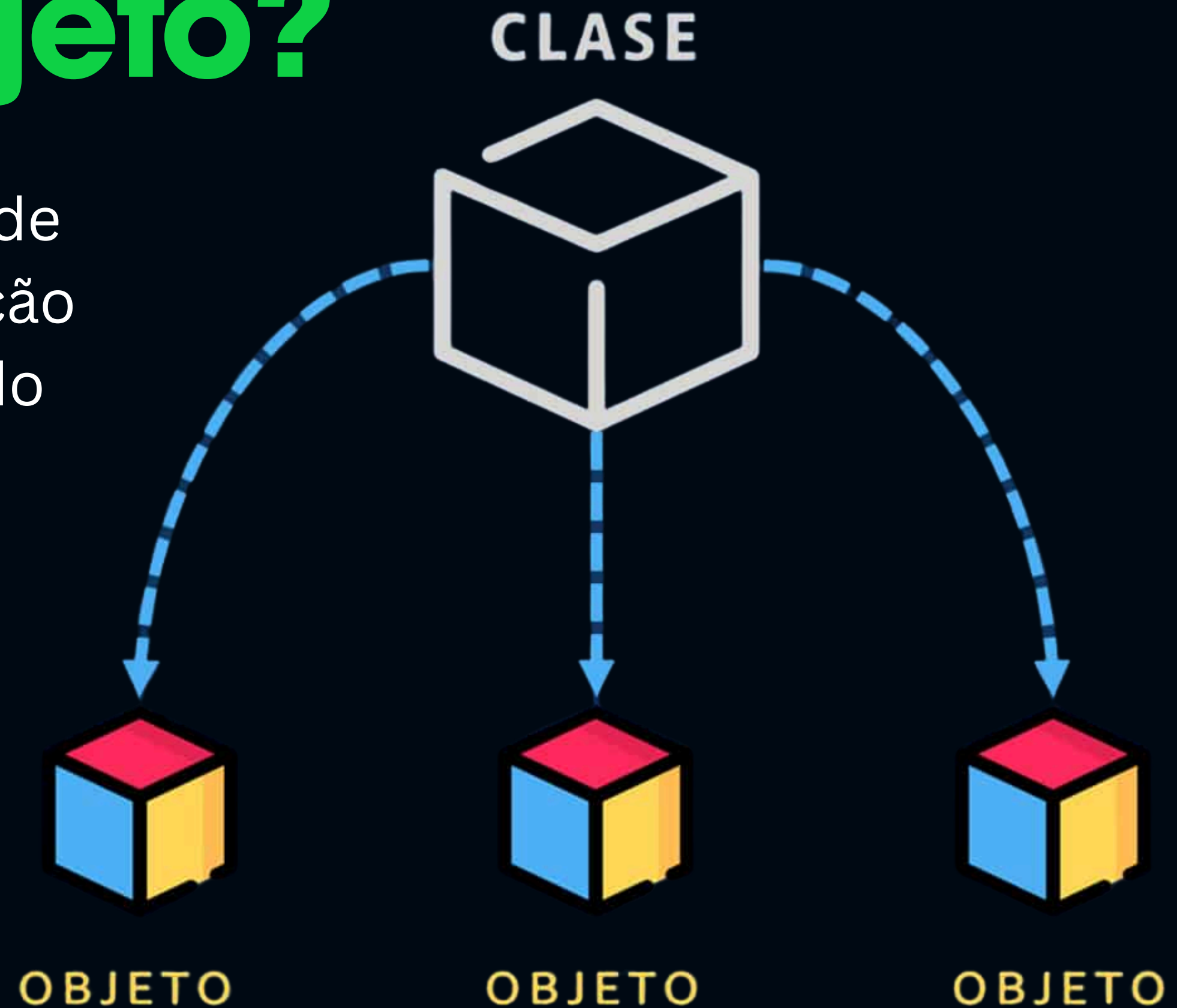
```
{ Chase-geast: A}
Coase_ccal: }
Chase_bire:
cothect: B11),
focal patted((ctagenalih'progantiin*))
focheete: glast*));
} }
```

Spring caser

```
{ Chase-grale: A}
Chase-geral )
Cuase-livl
Cnate: A1 }
contect: B10);
focteast getly'regentiint:'pecerobc'4*i)
focal detassint(),
} }
```

O que é um objeto?

Um objeto é uma instância concreta de uma classe, ou seja, uma representação real criada a partir do modelo definido pela classe.



O que é uma classe?

Em Java, uma classe é uma estrutura fundamental da Programação Orientada a Objetos (POO) que serve como um tipo de dado definido pelo programador.

Ela encapsula atributos (variáveis de instância) e métodos (funções ou comportamentos) que definem o estado e as ações dos objetos criados a partir dela.

```
4 inta) latfbate {
9 dill crate:
8 edth:
7 cet.les{ Eft0, Printe pfaci ayster"9,
9 vwpneautoiecam Potalzahon 2:11
9 afe-sasynciants #ICrleFowreciotals 7/88
10 ayyphesure Eyplante"4% _CoUl cuthem 77- Co " Fatth 73
11-oviess ataZe {
21 ceveltons(f)) Dife3
11 *Solonapewers ("Forianu.witec\Frum 2o 47% 0)5:2206{
12 "priis)>): 7/47
22 rapcketc{(): 71/3)
39 #ours ayfles("ate{)" ,
23 Reris { grarter llden, Saplt { 72/25
24 RYlatiynid Dir! Eatien,8)."6<leckil" co-guteon."762
29 snib( :
15 <d}2g/leprsat, 81/33
11 = The ["ici&2 { ""IB31D-perplalid") lita?
16 /.cesit |.cutret)"6
14 iacius ieplyce (, lattr/Onec(Flevo")
14 aryansentical, Cwls BIf1,dics
15 tesiustze) test ) 71/35
19 regeneen, nere"ow" _ficlage {} 71/23
19 fatah .Od
12 tat , etdl Iiyrranns", offer 71/21
18 COB2nuAnt:
```

➤ Criando Classes em Java



Professor: Carlos Sabino

// Classe Carro

```
public class Carro {
```

// Atributos (características do objeto)

```
String marca;
```

```
String modelo;
```

```
int ano;
```

// Método (comportamento do objeto)

```
void ligar() {
```

```
    System.out.println("O carro está ligando...");
```

```
}
```

```
void exibirInformacoes() {
```

```
    System.out.println("Marca: " + marca);
```

```
    System.out.println("Modelo: " + modelo);
```

```
    System.out.println("Ano: " + ano);
```

```
}
```

```
}
```

#POO em JAVA

➤ Instanciando Objetos em Java



Professor: Carlos Sabino

```
public class Main {  
    // Método principal para testar o objeto  
    public static void main(String[] args) {  
        // Criando um objeto da classe Carro  
        var meuCarro = new Carro();  
  
        // Atribuindo valores aos atributos do objeto  
        meuCarro.marca = "Toyota";  
        meuCarro.modelo = "Corolla";  
        meuCarro.ano = 2022;  
  
        // Chamando métodos do objeto  
        meuCarro.ligar();  
        meuCarro.exibirInformacoes();  
    }  
}
```


➤ Atributos

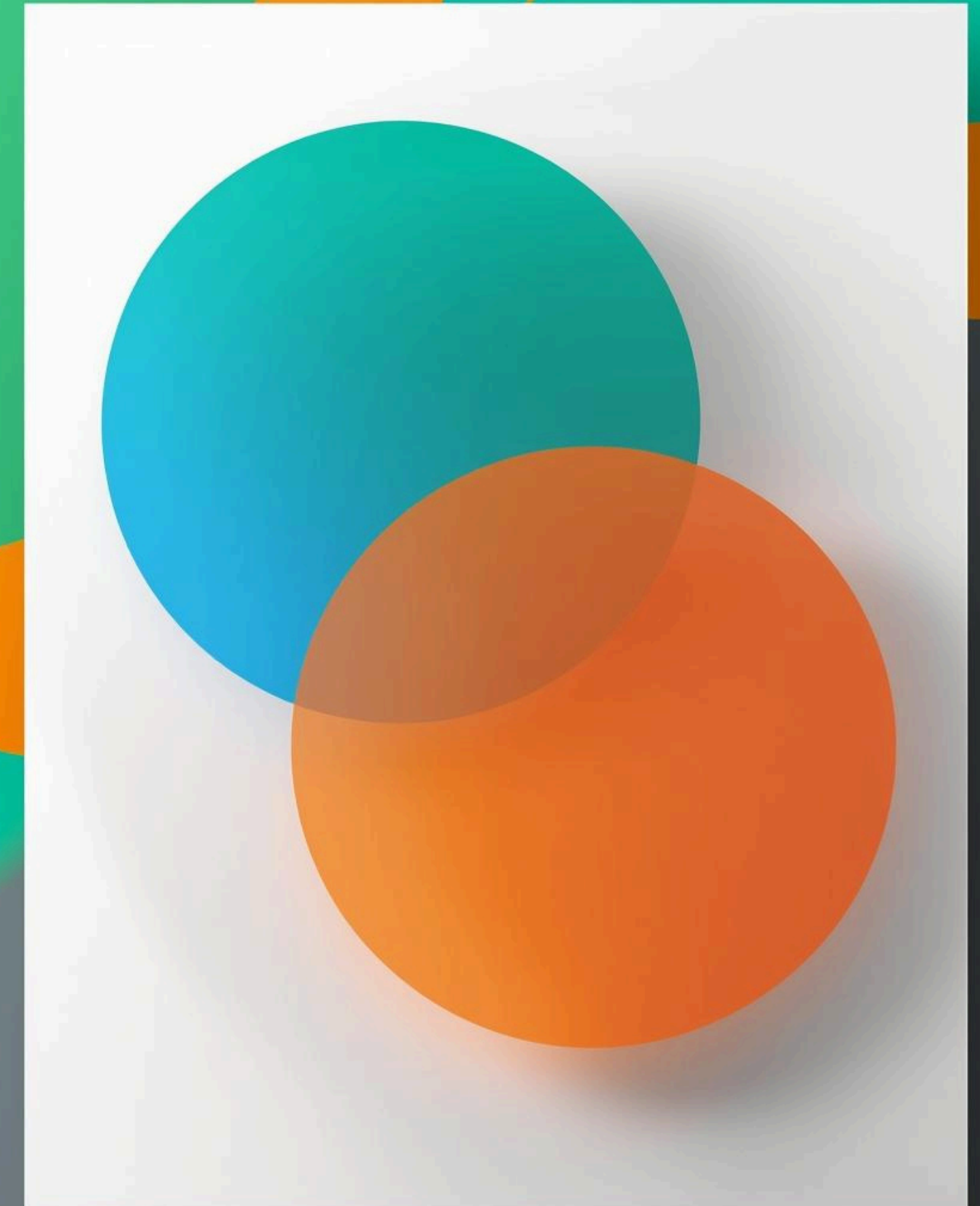
Os atributos são as características ou propriedades de uma classe.

Eles representam dados armazenados dentro de um objeto, que definem o seu estado.

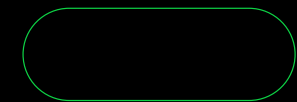


Professor: Carlos Sabino

```
public class Carro {  
    private String modelo;           // atributo privado  
    private String cor;              // atributo privado  
    private int ano;                 // atributo privado  
    protected double velocidade;    // atributo protegido  
    public boolean ligado;          // atributo público  
}
```

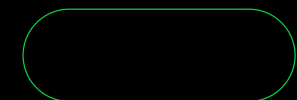


Pilares da Programação Orientada a Objetos



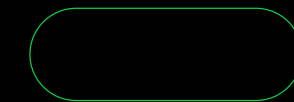
Abstração

Conceito fundamental para simplificar sistemas complexos.



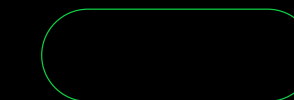
Herança

Permite reutilização e extensão de classes existentes.



Encapsulamento

Protege dados e oculta detalhes de implementação.



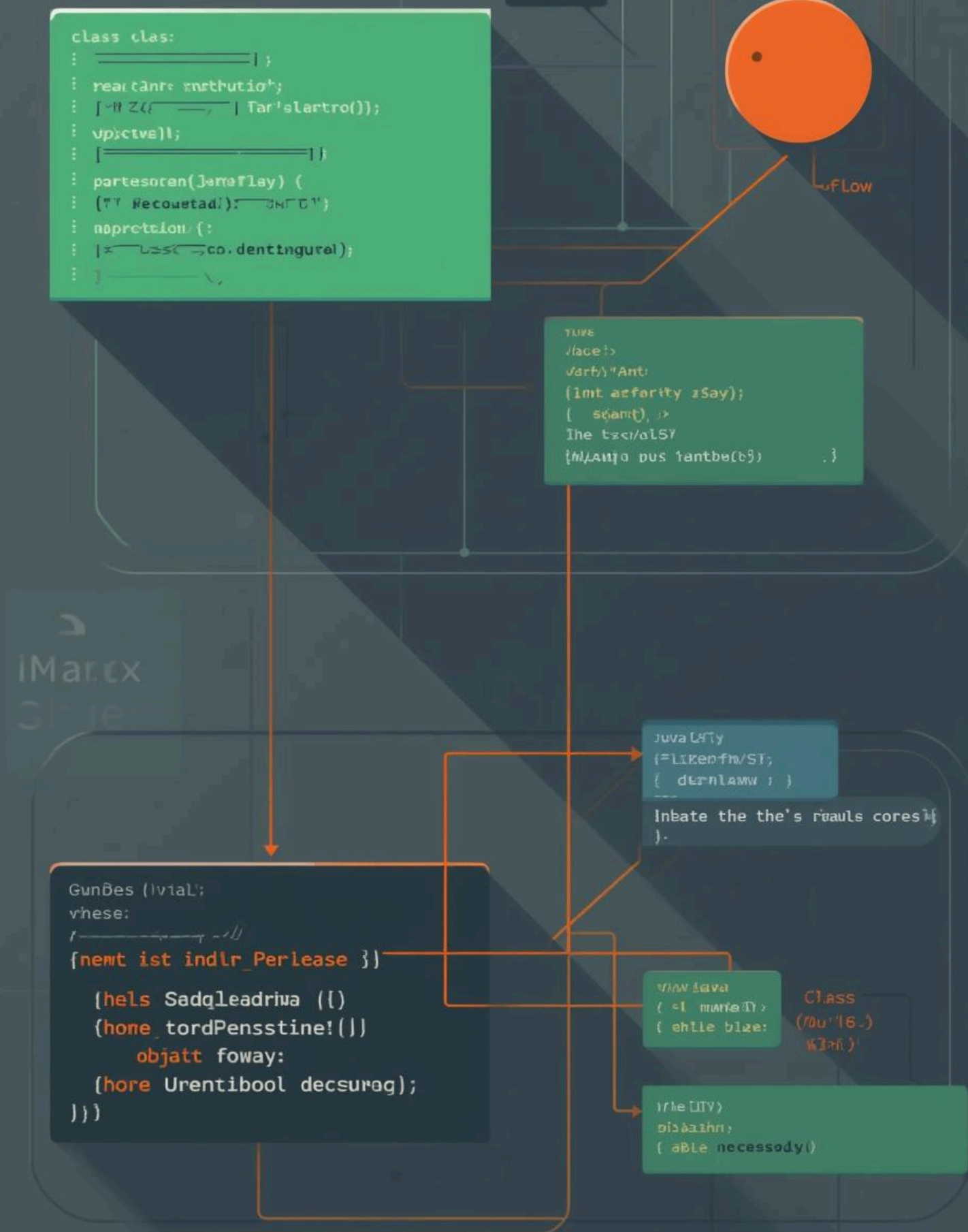
Polimorfismo

Facilita o uso de métodos de forma flexível.



Ao seguir esses pilares temos as garantias de

1. Segurança e Controle dos Dados (**Encapsulamento**)
2. Reutilização e Organização do Código (**Herança**)
3. Flexibilidade e Extensibilidade (**Polimorfismo**)
4. Clareza e Facilidade de Modelagem (**Abstração**)



Abstração

Simplificando a Complexidade

A abstração permite ocultar detalhes desnecessários, focando apenas no essencial. Imagine um carro: você utiliza o volante, mas não precisa saber como o motor funciona.



➤ Criando a abstração



Professor: Carlos Sabino

// Classe abstrata representando um conceito genérico de carro

```
public abstract class Carro {
```

```
    private String modelo;
```

```
    private String cor;
```

```
    private int ano;
```

// Construtor

```
    public Carro(String modelo, String cor, int ano) {
```

```
        this.modelo = modelo;
```

```
        this.cor = cor;
```

```
        this.ano = ano;
```

```
    }
```

// Método abstrato — cada tipo de carro deve implementar o seu próprio comportamento

```
    public abstract void acelerar();
```

// Getters e Setters

```
}
```

➤ Estendendo a abstração



Professor: Carlos Sabino

```
public class CarroEsportivo extends Carro {  
  
    public CarroEsportivo(String modelo, String cor, int ano) {  
        super(modelo, cor, ano);  
    }  
  
    // Implementando o método abstrato  
    @Override  
    public void acelerar() {  
        System.out.println("O carro esportivo acelerou rapidamente de 0 a 100 km/h!");  
    }  
}
```

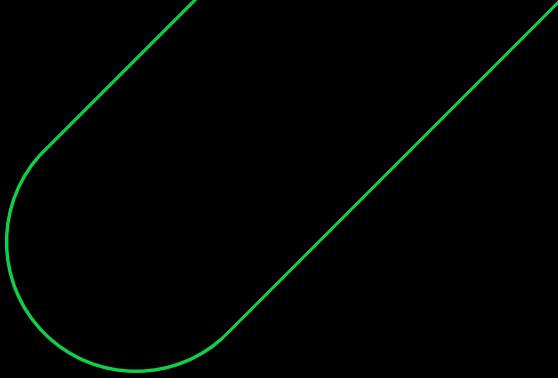
➤ Aplicando a abstração



Professor: Carlos Sabino

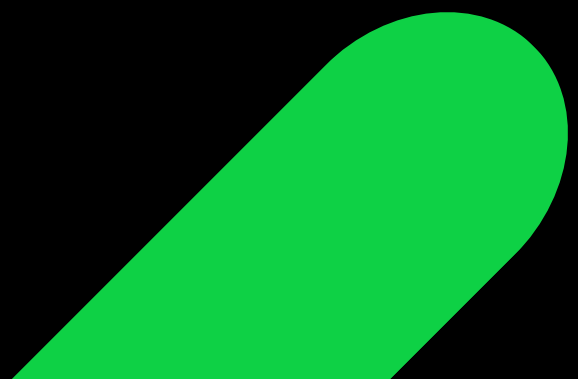
```
public class Main {  
    public static void main(String[] args) {  
  
        Carro carroEsportivo = new CarroEsportivo("Ferrari F8", "Vermelho", 2024);  
  
        carroEsportivo.acelerar();  
  
    }  
}
```

#POO em JAVA



“A abstração simplifica sistemas complexos, destacando as características essenciais e ocultando detalhes desnecessários.”

PROFESSOR: CARLOS SABINO



Vantagens da Abstração no desenvolvimento

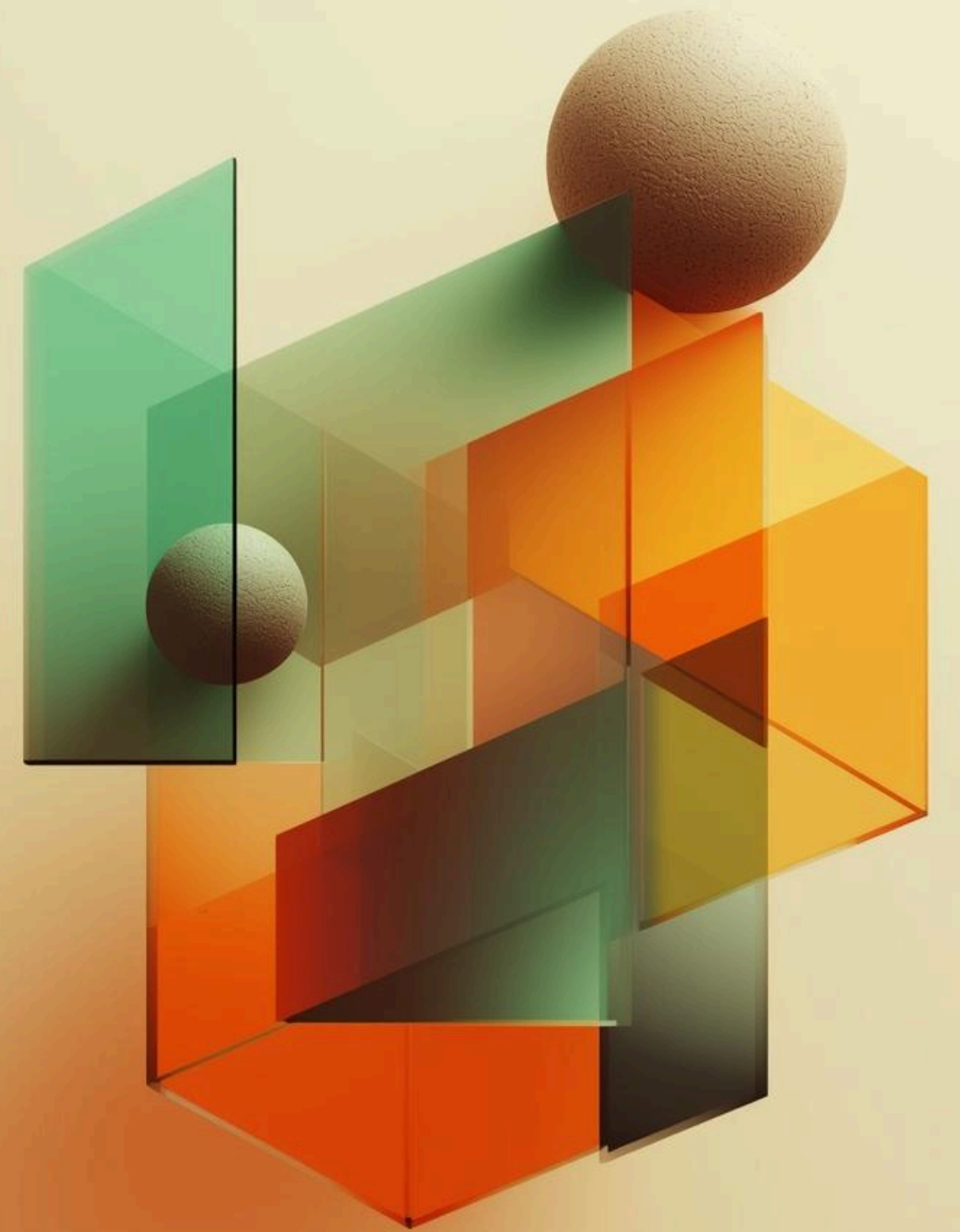
- Simplifica sistemas complexos em representações gerenciáveis
- Representa características essenciais, ocultando detalhes desnecessários
- Analogia: o painel de um carro mostra apenas as informações principais
- Trecho de código demonstrando abstração em uma classe Java
- Diagrama UML ilustrando relacionamentos entre classes e encapsulamento



Vamos praticar!

Crie uma classe abstrata Veiculo com:

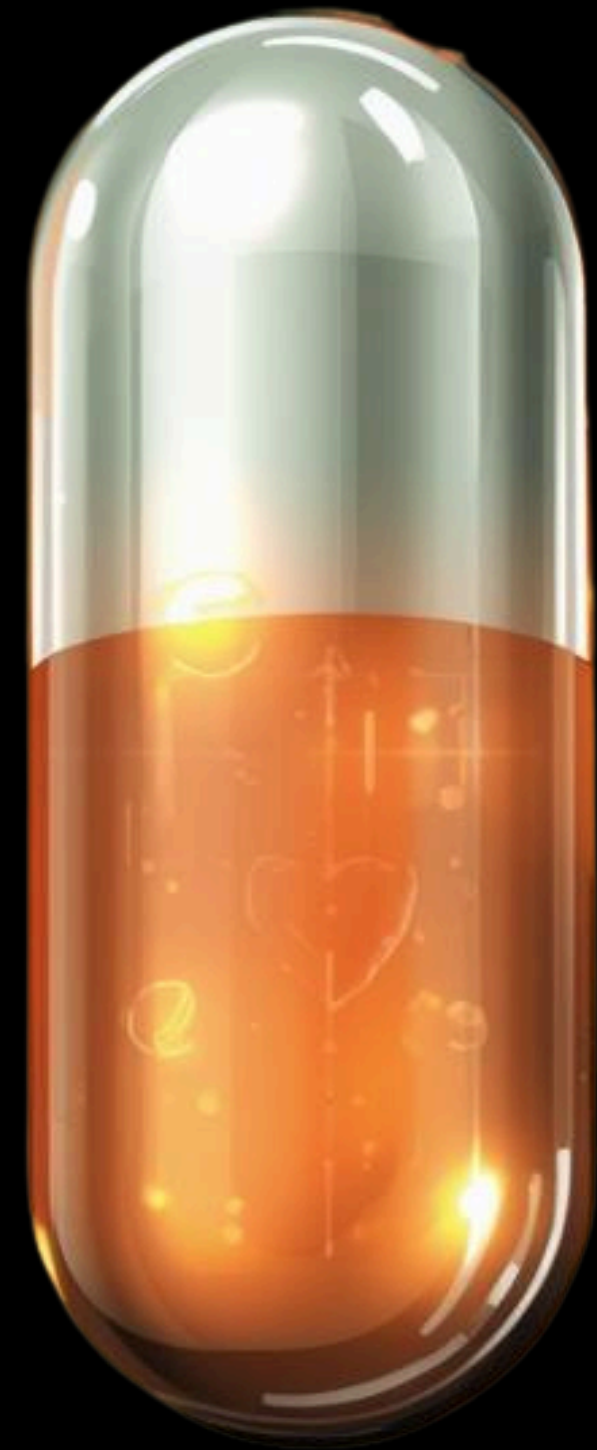
- **Atributos:** marca, modelo, ano.
- **Métodos:** acelerar(), frear(), abastecer().



Encapsulamento: Protegendo Seus Dados

Simplificando a Complexidade

O encapsulamento é a prática de proteger dados dentro de classes, garantindo acesso controlado. Por exemplo, uma cápsula armazena informações, semelhante a um objeto Java que usa métodos para manipular dados.



Encapsulando a aplicação



Professor: Carlos Sabino

```
public class ContaBancaria {  
    // Atributo encapsulado  
    private double saldo;  
  
    // Método para acessar o saldo (getter)  
    public double getSaldo() {  
        return saldo;  
    }  
  
    // Método para alterar o saldo (setter)  
    public void depositar(double valor) {  
        if (valor > 0) {  
            saldo += valor;  
        } else {  
            System.out.println("Valor inválido para depósito!");  
        }  
    }  
}
```

#POO em JAVA



Modificadores de Acesso em Java

Os modificadores de acesso em Java são palavras-chave usadas para controlar a visibilidade e o acesso a classes, atributos, métodos e construtores dentro de um programa. Eles fazem parte das boas práticas de encapsulamento, permitindo que você proteja dados e funcionalidades, definindo quem pode usar ou modificar cada parte do código.



private

public

protected

default

Modificadores de Acesso em Java

private

Onde pode ser acessado?

Apenas dentro da própria classe

Uso típico:

Atributos e métodos que devem ser protegidos



Modificadores de Acesso em Java

Onde pode ser acessado?

Qualquer classe

Uso típico:

Métodos ou classes que devem ser acessíveis de qualquer lugar

private

public



Modificadores de Acesso em Java

Onde pode ser acessado?

Mesma classe, subclasses e classes do mesmo pacote

Uso típico:

Permite herança e compartilhamento controlado



private

public

protected



Modificadores de Acesso em Java

Onde pode ser acessado?

Classes do mesmo pacote

Uso típico:

Visibilidade limitada ao pacote
sem palavra-chave



private

public

protected

default

MODIFICADORES DE ACESSO EM JAVA



MODIFICADOR	MESMA CLASSE	MESMO PACOTE	SUBCLASSE	TODA A APLICAÇÃO
public	✓	✓	✓	✓
protected	✓	✓	✓	✗
default	✓	✓	✗	✗
private	✓	✗	✗	✗

➤ Aplicando o Encapsulamento em Java com o Uso dos Modificadores de Acesso



Professor: Carlos Sabino

```
public class Pessoa {  
  
    private String nome;    // private → só pode ser acessado dentro da própria classe  
    String cpf;            // default → acessível apenas dentro do mesmo pacote  
    protected int idade;   // protected → acessível por subclasses e classes do mesmo pacote  
    public String cidade;  // public → acessível de qualquer lugar  
  
    // Criação dos métodos de acesso (getters) e modificação (setters)  
  
}
```

#POO em JAVA

Como aplicamos o encapsulamento em Java?

Usamos os modificadores de acesso (private, public, protected, e default) para controlar a visibilidade de atributos e métodos.

Depois, criamos métodos públicos (getters e setters) para permitir o acesso seguro.



Atividade: Encapsulamento

Conceitos e princípios fundamentais

Crie uma classe Pessoa com os atributos:

- nome (String)
- idade (int)
- cpf (String)

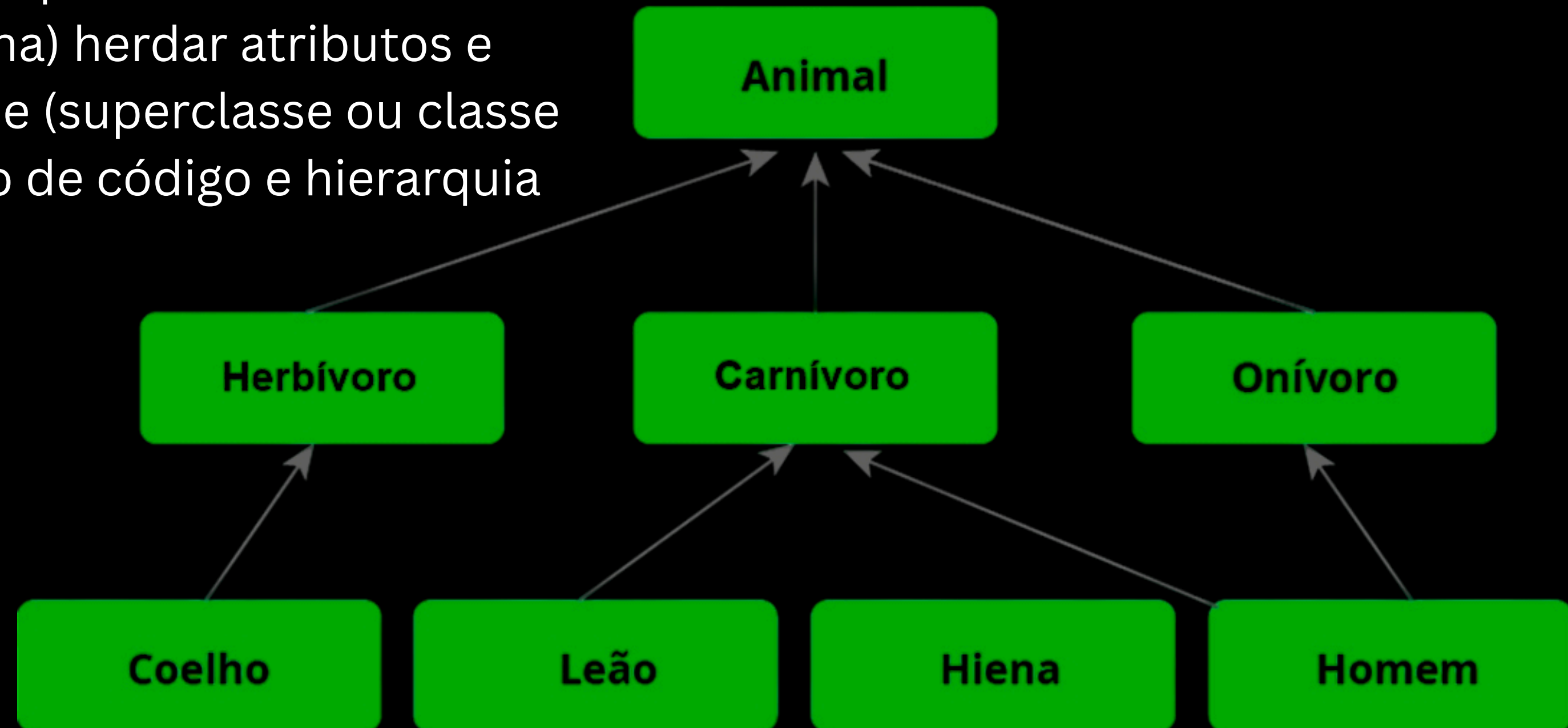
Critérios

1. Todos devem ser privados.
2. Crie os métodos getters e setters para acessar e modificar os valores.
3. No main, crie um objeto da classe e exiba os dados.

Herança

Simplificando a Complexidade

Herança é o mecanismo da Programação Orientada a Objetos que permite a uma classe (subclasse ou classe filha) herdar atributos e métodos de outra classe (superclasse ou classe pai), promovendo reuso de código e hierarquia entre classes.



➤ Sintaxe da herança em Java



Professor: Carlos Sabino

```
public class SuperClasse {  
    // Atributos e métodos  
}
```

```
public class SubClasse extends SuperClasse {  
    // Novos atributos e métodos ou sobrescrita de métodos  
}
```

#POO em JAVA

➤ Criando a Superclasse



Professor: Carlos Sabino

```
public class Veiculo {  
    protected String marca;  
    protected int ano;  
  
    public void ligar() {  
        System.out.println("Veículo ligado");  
    }  
}
```

#POO em JAVA

➤ Criando a Subclasse



Professor: Carlos Sabino

```
public class Carro extends Veiculo {  
    private int portas;  
  
    public Carro(String marca, int ano, int portas) {  
        this.marca = marca;  
        this.ano = ano;  
        this.portas = portas;  
    }  
  
    public void exibirInfo() {  
        System.out.println("Marca: " + marca + ", Ano: " + ano + ", Portas: " + portas);  
    }  
}
```

➤ Criando a Subclasse



Professor: Carlos Sabino

```
public class Main {  
    public static void main(String[] args) {  
        Carro carro = new Carro("Toyota", 2020, 4);  
        carro.ligar();           // Herdado da classe Veiculo  
        carro.exibirInfo();      // Específico da subclasse Carro  
    }  
}
```

#POO em JAVA

➤ Sobrescrita de métodos (Override)

A subclasse pode modificar o comportamento de métodos herdados da superclasse usando @Override.



Professor: Carlos Sabino

```
@Override  
public void ligar() {  
    System.out.println("Carro ligado com chave eletrônica!");  
}
```

#POO em JAVA

Vamos praticar!

Crie a classe base Funcionario com:

- Atributos: nome, salarioBase.
- Método: calcularSalario() que retorna o salário base.

Crie subclasses:

- Gerente (atributo: bonus) → salário total = salarioBase + bonus.
- Vendedor (atributo: comissao) → salário total = salarioBase + comissao.

```
4 inta) latfbate {
9 dill crate:
8 edth:
7 cet.les{ Eft0, Printe pfaci ayster "9,
9 vwpneautoiecam Potalzahon 2:11
9 aBw-sasynciants #ICrleFomreciotals 7/88
10 ayyphesure Erplante "4% _CoU cuthem 77- Co " Fatth 73
11-oviess ataZe {
21 ceveltons(f)) Dife3
11 *Solonapewers ("Forianu.witec\Frum 2o 47% 0)5:2206{
12 "priis)>): 7/47
22 rapcketc{()}; 7i/3)
39 #ours ayfiles("ate{)" ,
23 Reris { grarter llden, Saplt { 72/25
24 RYlatiynid Dir! Eatien,8)."6<leckil" co-guteon. "762
29 snib( :
15 <dj2g/leprsat, 81/3d
11 = The ["ici02 { ""IB31D-perplalid") lita?
16 /.cesit |.cutret)"6
14 iacius ieplyce (, lattp/Onec(Flevo")
14 aryansentical, Cwls Bifl,dics
15 tesiustze) test ) 7i/35
19 regeneen, nere"ow" _ficlage {} 71/23
19 fatah .Od
12 tat , etdl Iiyrapns", offer 71/21
18 COB2nuAnt:
```

Polimorfismo

Simplificando a Complexidade

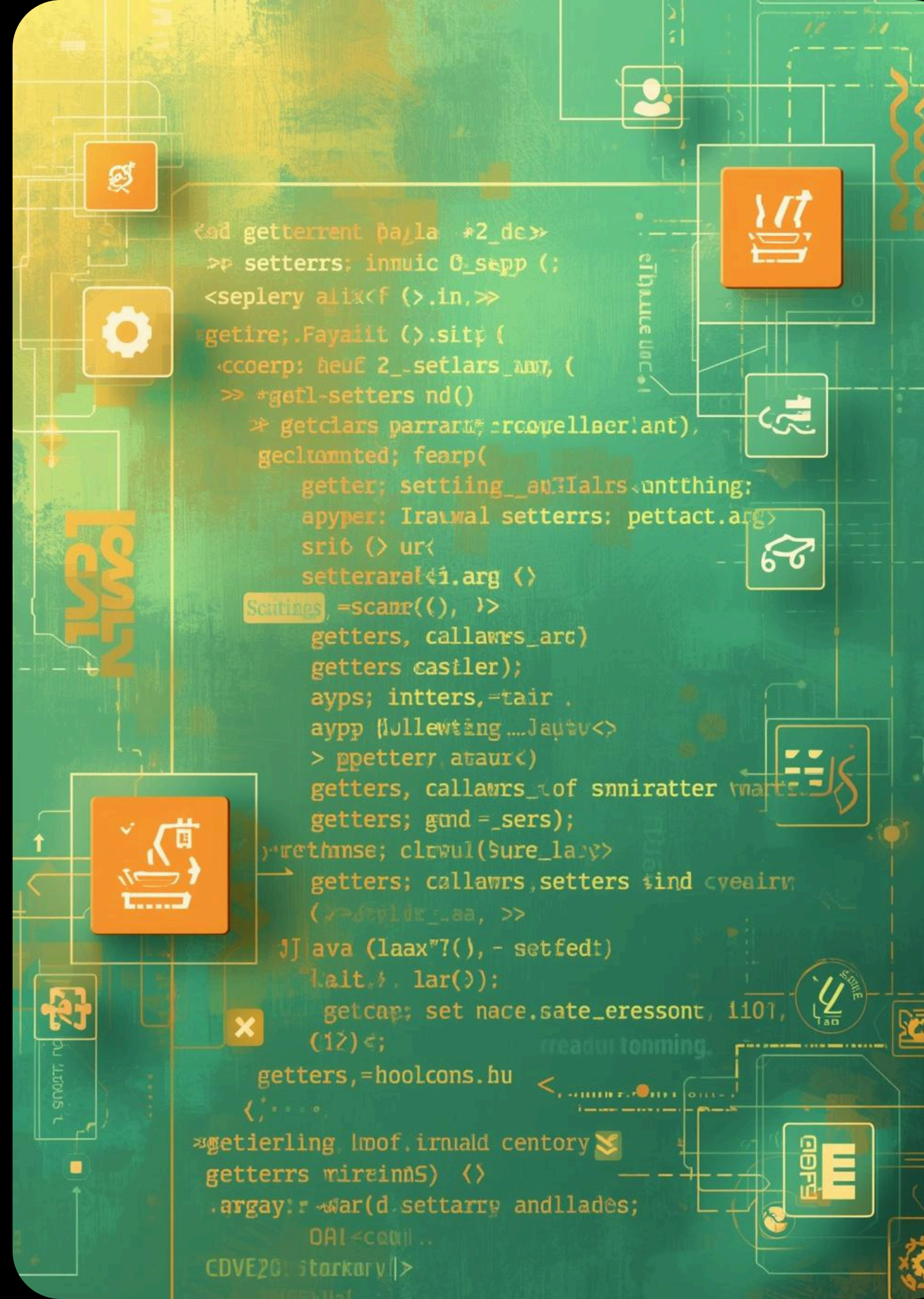
Polimorfismo é um mecanismo que permite que objetos de diferentes classes sejam tratados de forma uniforme por meio de uma mesma interface ou superclasse, mas que apresentem comportamentos distintos em tempo de execução.



Tipos de Polimorfismo em Java

Polimorfismo de Sobrecarga

Ocorre quando métodos têm o mesmo nome, mas diferentes assinaturas (quantidade ou tipo de parâmetros). É resolvido em tempo de compilação.



```
<ad getterrent pa/la #2_de>>  
>> setterr; inmuic 0_sapp (:  
<seplery alix<f (>.in,>>  
getire;.Fayait (>.sitp (  
<coerp: heuf 2_-setlars_amy, (  
>> #getl-setters nd()  
>> getclars parran; rrcovellerlant),  
geclumnted; fearp(  
getter; settiing_au3Ialrs<untthing:  
apyper: Iraimal setterr; pettact.arg>  
srib (> ur<  
setteraral<1.arg (>  
Settings,=scanr(<), >>  
getters, callawrs_arc)  
getters castler);  
ayps; intters,=tair .  
aypp (Jullewtang ....Japuv<>  
> ppetterr ataur<>  
getters, callawrs_of smniratter warts  
getters; gmd=_sers);  
>>rethmse; clrwul(Sure_lay>  
getters; callawrs,settters find cyearw  
<>>Jeylax_ua, >>  
JJava (laax"7(<), - setfedt)  
lalt,; lar(<);  
getcap; set nace.sate_erssont, 1101,  
(12)<;  
getters,=hoolcons.hu  
<.....  
>>getierling, lmof,irnuald centory  
getters wirinns) (<)  
.argay: r-war(d settarrr andllades;  
OAI<catu1..  
CDVE201 storkory||>
```

➤ Usando o Polimorfismo de Sobrecarga



Professor: Carlos Sabino

```
public class Calculadora {  
  
    // Método 1 – soma de dois inteiros  
    public int somar(int a, int b) {  
        return a + b;  
    }  
  
    // Método 2 – soma de três inteiros  
    public int somar(int a, int b, int c) {  
        return a + b + c;  
    }  
  
    // Método 3 – soma de dois números decimais  
    public double somar(double a, double b) {  
        return a + b;  
    }  
}
```

#POO em JAVA

➤ Aplicando o Polimorfismo de Sobrecarga



Professor: Carlos Sabino

```
public class Teste {  
    public static void main(String[] args) {  
        Calculadora calculadora = new Calculadora();  
  
        System.out.println(calculadora.somar(2, 3));    // Usa o método int,int  
        System.out.println(calculadora.somar(2, 3, 4)); // Usa o método int,int,int  
        System.out.println(calculadora.somar(2.5, 3.7)); // Usa o método double,double  
    }  
}
```

#POO em JAVA

Tipos de Polimorfismo em Java

Polimorfismo de Sobreescrita

Também chamado de polimorfismo em tempo de execução, ocorre quando uma subclasse redefine (sobrescreve) um método que já existe na superclasse.

```
<ad getterrent pa/la #2_de>
>> setterr; inmuic 0_sapp (
<seplery alix<f (>.in,>>
getire;.Fayait (>.sitp (
<coerp: heuf 2_-setlars_auy, (
>> #getl-setters nd()
>> getclars parran; rrcovellerlant),
geclumnted; fearp(
getter; settiing_au3lals<untthing:
apyper: Iraimal setterr; pettact.arg>
srib (> ur<
setteraral<1.arg (>
Settings,=scanr(<>, )>
getters, callaws_arc)
getters castler);
ayps; intters,=tair .
aypp (lullewing ....Jauu<>
> ppeterr; ataur<
getters, callaws_of smniratter (watts
getters; gmd=_sers);
>>rethmse; clrwul(Sure_lauy>
getters; callaws,settters find cyearw
(>>Jeylax_ua, >>
JJ ava (laax"7(<>, - setfedt)
lalt,; lar(<>);
getcap; set nace.sate_erssont, 1101,
(12)<;
getters,=hoolcons.hu
<.....
>>getierling, lmof,irnuald centory
getters wirinns) (<>
.argay: r-war(d settarpy andllades;
OAI<catu...
CDVE201 storkory||>
```

➤ Usando o Polimorfismo de sobrescrita



Professor: Carlos Sabino

// Classe base (superclasse)

```
class Animal {  
    public void emitirSom() {  
        System.out.println("O animal faz um som...");  
    }  
}
```

// Subclasse 1

```
class Cachorro extends Animal {  
    @Override  
    public void emitirSom() {  
        System.out.println("O cachorro late: Au Au!");  
    }  
}
```

// Subclasse 2

```
class Gato extends Animal {  
    @Override  
    public void emitirSom() {  
        System.out.println("O gato mia: Miau!");  
    }  
}
```

#POO em JAVA

➤ Aplicando Polimorfismo de sobrescrita



Professor: Carlos Sabino

// Classe principal

```
public class Main {
```

```
    public static void main(String[] args) {
```

```
        Animal a1 = new Cachorro(); // polimorfismo
```

```
        Animal a2 = new Gato();
```

```
        a1.emitirSom(); // Chama o método da classe Cachorro
```

```
        a2.emitirSom(); // Chama o método da classe Gato
```

```
    }
```

```
}
```

#POO em JAVA

Regras para sobrescrita

```
class Animal {  
    public void emitirSom() {  
        System.out.println("O animal faz um som...");  
    }  
}
```

```
class Cachorro extends Animal {  
    @Override  
    public void emitirSom() {  
        System.out.println("O cachorro late: Au Au!");  
    }  
}
```

O método deve ter mesmo nome e parâmetros da superclasse.

O tipo de retorno deve ser igual ou covariante (compatível).

O método da superclasse deve ser visível (não private).

É comum usar a anotação `@Override` para indicar a sobrescrita.

Regras para sobrescrita

```
class Animal {  
    public void emitirSom() {  
        System.out.println("O animal faz um som...");  
    }  
}
```

```
class Cachorro extends Animal {  
    @Override  
    public void emitirSom() {  
        System.out.println("O cachorro late: Au Au!");  
    }  
}
```

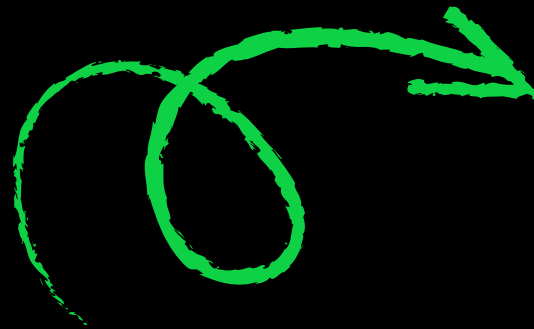


O método deve ter mesmo nome e parâmetros da superclasse.

Regras para sobrescrita

```
class Animal {  
    public void emitirSom() {  
        System.out.println("O animal faz um som...");  
    }  
}
```

```
class Cachorro extends Animal {  
    @Override  
    public void emitirSom() {  
        System.out.println("O cachorro late: Au Au!");  
    }  
}
```



O método deve ter mesmo nome e parâmetros da superclasse.

O tipo de retorno deve ser igual ou covariante (compatível).

Regras para sobrescrita

```
class Animal {  
    public void emitirSom() {  
        System.out.println("O animal faz um som...");  
    }  
}
```

```
class Cachorro extends Animal {  
    @Override  
    public void emitirSom() {  
        System.out.println("O cachorro late: Au Au!");  
    }  
}
```



O método deve ter mesmo nome e parâmetros da superclasse.

O tipo de retorno deve ser igual ou covariante (compatível).

O método da superclasse deve ser visível (não private).

Regras para sobrescrita

```
class Animal {  
    public void emitirSom() {  
        System.out.println("O animal faz um som...");  
    }  
}
```

```
class Cachorro extends Animal {  
    @Override  
    public void emitirSom() {  
        System.out.println("O cachorro late: Au Au!");  
    }  
}
```



O método deve ter mesmo nome e parâmetros da superclasse.

O tipo de retorno deve ser igual ou covariante (compatível).

O método da superclasse deve ser visível (não private).

É comum usar a anotação @Override para indicar a sobrescrita.

Interfaces em Java

Uma interface em Java é um contrato que define métodos que uma classe deve implementar.

interface

```
interface  
class: final = default  
= final;  
namespace f  
    final static:  
    withcardinalities by  
    direction>  
    "Arithmetic">  
    <size= 12 source>  
    content-label>  
    <order = (boolean (Act))>  
    <face: possible>>  
>> implement>
```

main/imports

```
import:  
class: final = default  
= final;  
= final;  
cdor final:  
    cime: busline>  
Implement (Subes ())  
    infretor: final>  
IMitot (Lapilt (it))  
    celice= 13 decauctly>  
    <face-inc-ch>>  
    <iterpe (focycle)>>  
    commet pilbbby>  
>> implthace>
```

➤ Aplicando o uso de Interfaces



Professor: Carlos Sabino

```
public interface Animal {  
    void emitirSom(); // método abstrato (não tem corpo)  
}
```

```
public class Cachorro implements Animal {  
    @Override  
    public void emitirSom() {  
        System.out.println("Au Au!");  
    }  
}
```

#POO em JAVA

Vamos praticar!

Implemente um programa em Java que simule o cálculo de imposto sobre produtos utilizando o conceito de interfaces em Programação Orientada a Objetos (POO).

Crie uma interface chamada `Tributavel`, que deverá conter:

- Uma constante `double TAXA = 0.15`; representando a taxa de imposto (15%).
- Um método abstrato `double calcularImposto()`; que será implementado pelas classes concretas.

Em seguida, crie uma classe chamada `Produto` com os seguintes atributos:

- `String nome`
- `double preco`

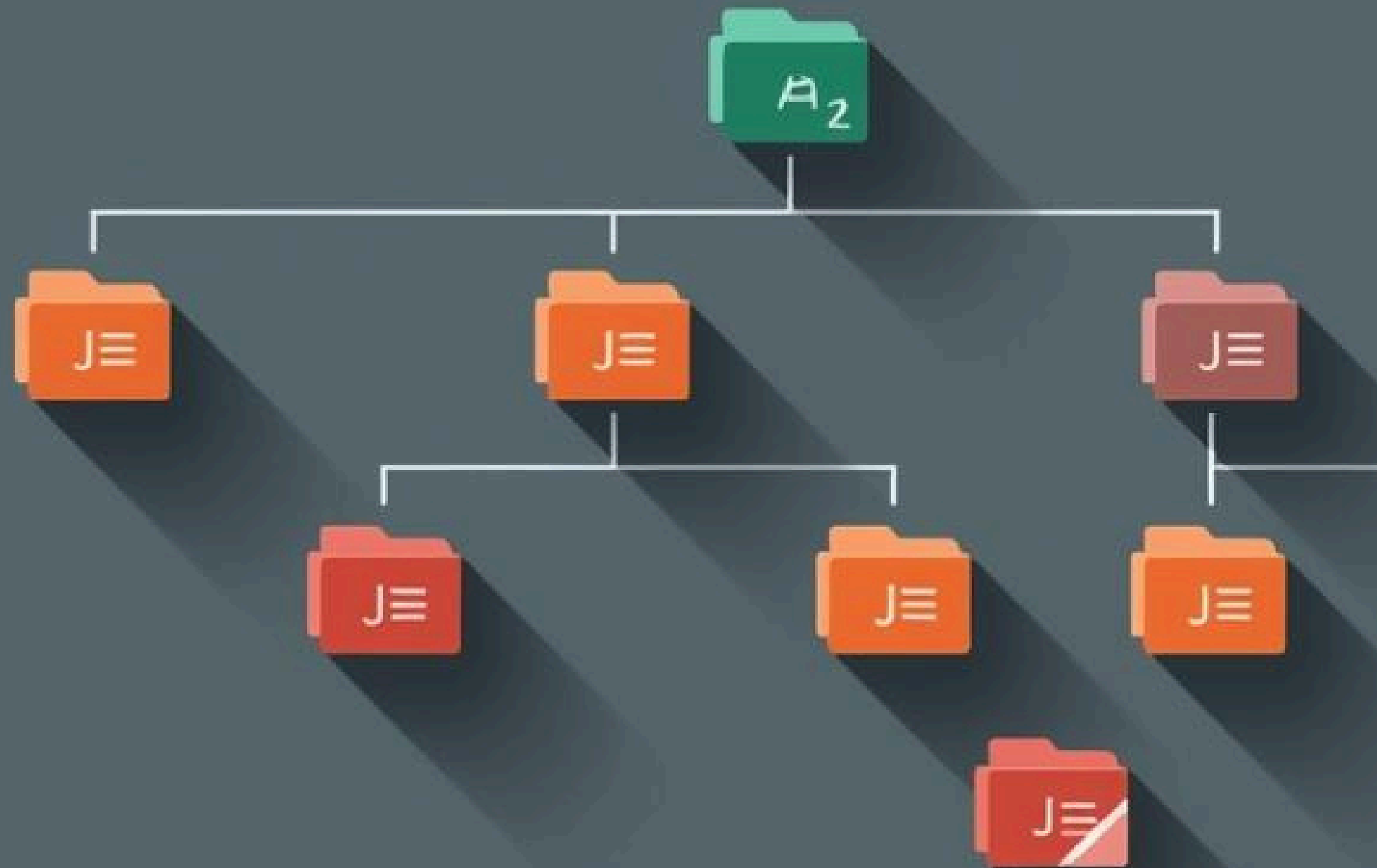
A classe `Produto` deve implementar a interface `Tributavel` e realizar o cálculo do imposto no método `calcularImposto()`, retornando o valor do imposto com base no preço do produto e na taxa definida.

Por fim, no método `main`, instancie um objeto da classe `Produto`, atribua um nome e um preço, calcule o imposto e exiba na tela:

- O nome do produto
- O preço original
- O valor do imposto
- O valor total (preço + imposto)

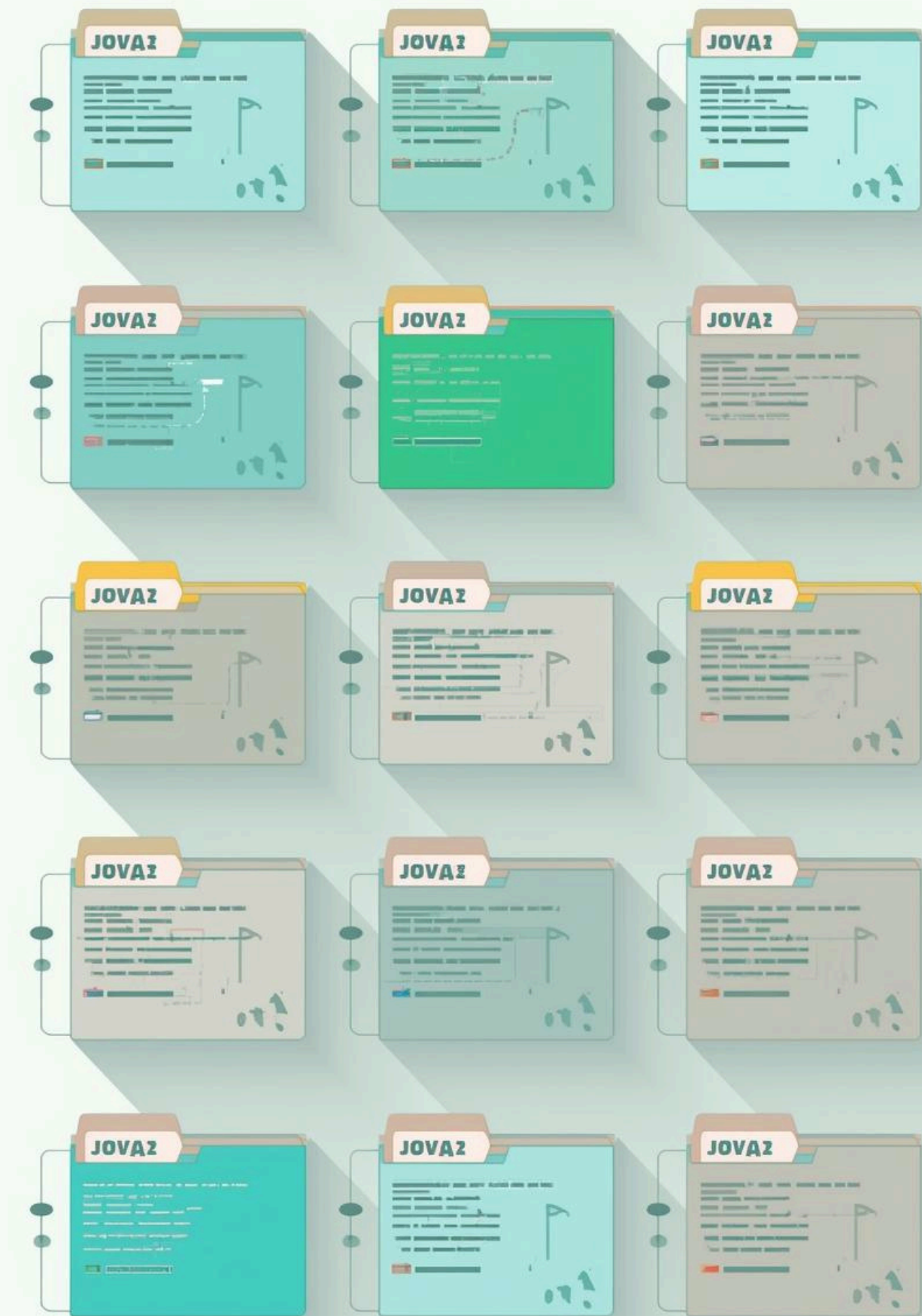
Modularização

Modularização é o processo de dividir um sistema em módulos independentes e coesos, geralmente representados por classes, pacotes e interfaces, para facilitar o desenvolvimento, a manutenção e a reutilização do código.

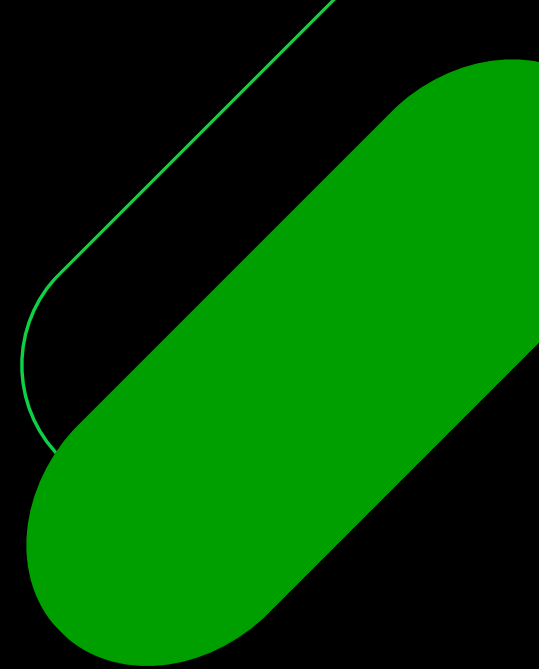


Pacotes em Java

```
api-usuario/  
├── module-info.java  
├── br/  
│   ├── usuario/  
│   │   ├── model/  
│   │   │   ├── Usuario.java  
│   │   ├── service/  
│   │   │   ├── UsuarioService.java  
│   │   ├── app/  
│   │   │   ├── Main.java
```



Vamos revisar os 4 pilares da POO



Abstração

Foca no essencial do objeto, ignorando detalhes internos complexos.

Encapsulamento

Protege os dados internos de um objeto, restringindo o acesso direto a eles e expondo apenas o que for necessário por meio de métodos (getters e setters).

Herança

Permite que uma classe (subclasse) herde atributos e métodos de outra classe (superclasse).

Polimorfismo

Capacidade de um objeto se comportar de diferentes maneiras dependendo do contexto.

Elementos Fundamentais da Programação Orientada a Objetos em Java



Classes

Classes são modelos para a criação de objetos.



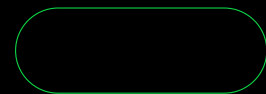
Objetos

Um objeto é uma instância de uma classe, ou seja, é a materialização na memória de um modelo (classe).



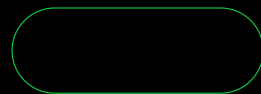
Instâncias

representam entidades únicas provenientes de classes.



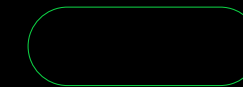
Interfaces

Interfaces definem contratos para as classes que as implementam.



Pacotes

Os pacotes organizam classes e interfaces relacionadas.



Métodos

Os métodos definem o comportamento dos objetos.

SOLID

As cinco letras do SOLID representam princípios de design que tornam o código mais limpo, flexível, fácil de manter e preparado para crescimento.

Eles são fundamentais para qualquer desenvolvedor Java, especialmente quem trabalha com APIs REST, microsserviços e aplicações profissionais.

**S – Single Responsibility Principle
(SRP)**

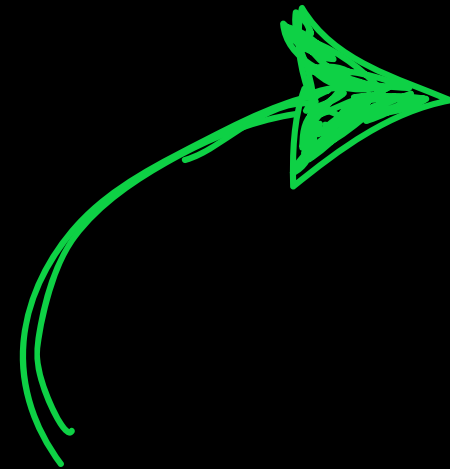
O – Open/Closed Principle (OCP)

**L – Liskov Substitution Principle
(LSP)**

**I – Interface Segregation Principle
(ISP)**

**D – Dependency Inversion Principle
(DIP)**

SOLID



**S – Single Responsibility Principle
(SRP)**

Princípio da Responsabilidade Única.
Cada classe deve ter uma única
responsabilidade, ou seja, um único
motivo para mudar.

➤ Princípio da Responsabilidade Única



Professor: Carlos Sabino

```
public class PedidoService {  
    public void salvarPedido(Pedido pedido) { ... }  
}
```

```
public class EmailService {  
    public void enviarEmailConfirmacao(Pedido pedido) { ... }  
}
```

```
public class NotaFiscalService {  
    public void gerarNotaFiscal(Pedido pedido) { ... }  
}
```


SOLID

**S – Single Responsibility Principle
(SRP)**

O – Open/Closed Principle (OCP)

Aberto para extensão e fechado para
modificação.

Você deve poder adicionar
funcionalidades sem alterar código já
existente.

➤ Princípio do Aberto para extensão e fechado



Professor: Carlos Sabino

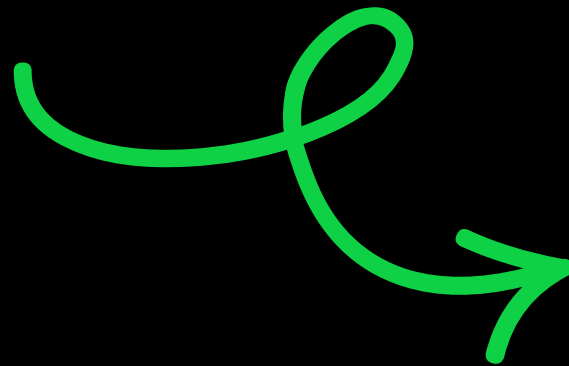
```
public interface Desconto {  
    double calcular(Produto produto);  
}
```

```
public class DescontoLivro implements Desconto {  
    public double calcular(Produto produto) { return produto.getValor() * 0.10; }  
}
```

```
public class DescontoEletronico implements Desconto {  
    public double calcular(Produto produto) { return produto.getValor() * 0.05; }  
}
```

SOLID

Princípio da Substituição de Liskov.
Uma classe filha deve poder substituir a classe pai sem quebrar a aplicação.



**S – Single Responsibility Principle
(SRP)**

O – Open/Closed Principle (OCP)

**L – Liskov Substitution Principle
(LSP)**

➤ Princípio da Substituição de Liskov



Professor: Carlos Sabino

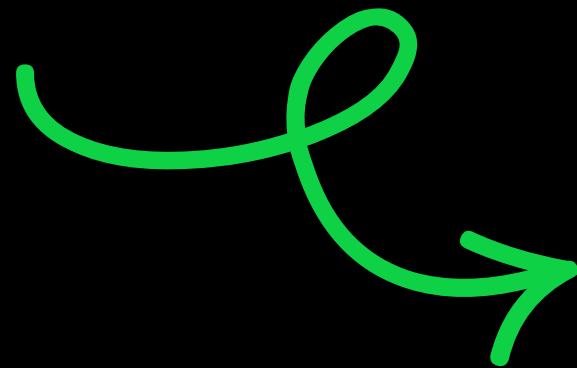
```
abstract class Pagamento {  
    public abstract void processar(double valor);  
}
```

```
class PagamentoPix extends Pagamento {  
    @Override  
    public void processar(double valor) {  
        System.out.println("Processando PIX no valor de R$ " + valor);  
    }  
}
```

```
class PagamentoCartaoCredito extends Pagamento {  
    @Override  
    public void processar(double valor) {  
        System.out.println("Processando cartão de crédito no valor de R$ " + valor);  
    }  
}
```

SOLID

Princípio da Segregação de Interfaces.
Nenhum cliente deve ser forçado a depender de métodos que não utiliza.



**S – Single Responsibility Principle
(SRP)**

O – Open/Closed Principle (OCP)

**L – Liskov Substitution Principle
(LSP)**

**I – Interface Segregation Principle
(ISP)**

➤ Princípio da Segregação de Interfaces.



Professor: Carlos Sabino

```
interface EmailNotificacao {  
    void enviarEmail(String msg);  
}
```

```
interface SMSNotificacao {  
    void enviarSMS(String msg);  
}
```

```
interface WhatsAppNotificacao {  
    void enviarWhatsApp(String msg);  
}
```

➤ Princípio da Segregação de Interfaces.



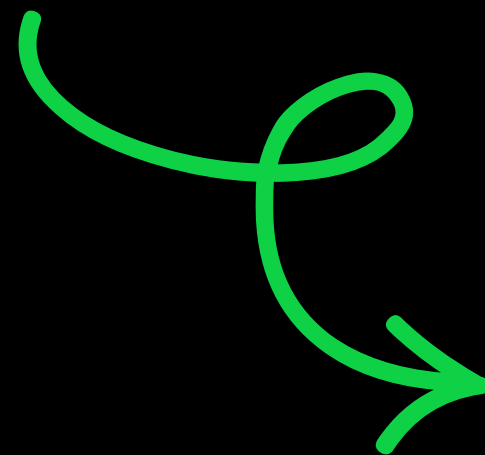
Professor: Carlos Sabino

```
class NotificacaoEmail implements EmailNotificacao {  
    @Override  
    public void enviarEmail(String msg) {  
        System.out.println("Email enviado: " + msg);  
    }  
}
```

```
class NotificacaoSMS implements SMSNotificacao {  
    @Override  
    public void enviarSMS(String msg) {  
        System.out.println("SMS enviado: " + msg);  
    }  
}
```

SOLID

Princípio da Inversão de Dependência
Classes de alto nível devem depender de abstrações, e não de implementações concretas.



S – Single Responsibility Principle (SRP)

O – Open/Closed Principle (OCP)

L – Liskov Substitution Principle (LSP)

I – Interface Segregation Principle (ISP)

D – Dependency Inversion Principle (DIP)

➤ Princípio da Inversão de Dependência



Professor: Carlos Sabino

```
public interface Notificacao {  
    void enviar();  
}
```

```
public class EmailService implements Notificacao {  
    public void enviar() { ... }  
}
```

➤ Princípio da Inversão de Dependência



Professor: Carlos Sabino

```
public class PedidoService {  
    private final Notificacao notificacao;  
  
    public PedidoService(Notificacao notificacao) {  
        this.notificacao = notificacao;  
    }  
  
    public void confirmarPedido() {  
        notificacao.enviar();  
    }  
}
```


Programação Orientada a Objetos

Professor: Carlos Sabino

